23ADR506 – DEEP LEARNING

ASSIGNMENT-I

PRIVACY POLICY SIMPLIFIER

Submitted by

JENILIA KAREN A – 717823I216

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

2025 - 2026

TABLE OF CONTENT

ABSTRACT

Understanding privacy policies is often a daunting task for the average user due to their length and legal language. This project aims to bridge the gap between technical legal documents and public understanding by using Natural Language Processing (NLP). The Privacy Policy Simplifier allows users to upload a PDF or paste raw text of a privacy policy. Using advanced NLP models like BART for summarization and DistilBERT for question-answering, the system extracts the core information from the document and responds to user queries. Built as a Flask-based web application, the tool ensures accessibility via a browser. By translating complex policy language into simplified summaries and offering an interactive Q&A interface, this system enhances user awareness and decision-making when it comes to data sharing and consent. This project demonstrates the application of AI in increasing digital transparency and user empowerment in the digital era.

PROJECT DESCRIPTION

Objective

- To design an AI-powered tool that simplifies complicated privacy policy documents using NLP, making them easier for users to understand effectively.
- To develop a system that extracts key information from uploaded policies and converts legal terms into readable, user-friendly language automatically.
- To enable users to ask specific questions about privacy policies and receive accurate, contextual answers generated from the uploaded content.
- To bridge the gap between technical legal documentation and public understanding, especially in the context of data protection and user consent.
- To support transparency and informed decision-making by helping users clearly know what personal data is collected, used, and shared by services.

Features

- Users can upload privacy policies in PDF format or manually input raw text to simplify and query through an intuitive interface.
- The system uses PyMuPDF to extract readable and structured text content from complex and multi-page PDF policy documents uploaded by users.
- Integrated summarization using BART model helps convert lengthy and legal-heavy privacy policies into short, clear, and user-friendly summaries automatically.
- A question-answering module powered by DistilBERT enables users to ask specific queries about the privacy policy and receive direct answers.

- The application displays both the original and simplified text, allowing users to compare the actual policy with the summarized version side-by-side.
- Built-in error handling ensures smooth operation, letting users know if file formats are unsupported or inputs are missing essential information.
- The web interface is responsive, lightweight, and compatible with modern browsers, offering seamless performance across desktop and mobile platforms.

Tools and Technologies Used

- **Flask:** A lightweight Python web framework used to create the backend server for handling form submissions and routing.
- **HTML5 and Bootstrap:** Used to design the front-end user interface with responsive layout, forms, and buttons for better usability.
- **PyMuPDF (fitz):** A Python library for extracting text from PDF files, including multi-page and layout-rich documents.
- **Hugging Face Transformers:** A powerful NLP library providing access to pre-trained models for summarization and question answering tasks.
- **BART Model:** A sequence-to-sequence transformer model used for generating simplified summaries of complex privacy policy text.
- **DistilBERT Model:** A distilled version of BERT, used for answering user queries based on the uploaded privacy policy.
- **PyTorch:** A deep learning framework used as the backend engine for running transformer models within the Hugging Face library.
- **Jinja2 Templating Engine:** Integrated with Flask to render dynamic HTML pages that display input, summaries, and answers interactively.
- **Werkzeug & WSGI Middleware:** Bundled with Flask to manage HTTP requests, file uploads, and session management in the application.

Working Mechanism

1. **User Input Interface**
   The application provides a web interface where users can either upload a PDF file or paste raw privacy policy text directly.
2. **File Upload or Text Processing**
   If a PDF is uploaded, it is temporarily saved to the server. PyMuPDF is used to extract readable text from each page of the document.
3. **Text Cleaning and Chunking**
   The extracted or pasted text is cleaned and split into chunks of manageable size (up to 1024 tokens) to suit model input limits.

4. **Summarization Process**
   Each chunk of text is passed through the BART transformer model, which generates a simplified, human-readable summary for that section.
5. **Summary Compilation**
   All the generated summaries from individual chunks are combined to form a complete, simplified version of the original privacy policy.
6. **User Question Input**
   Users can enter specific questions related to the policy, such as "What data is collected?" or "Is data shared with third parties?"
7. **Question Answering**
   The DistilBERT model takes the full original text and the user's question as input, returning a relevant, context-aware answer.
8. **Result Display**
   Both the simplified summary and the answer to the question are displayed in the browser for the user to review.

Applications

- Beneficial for users who want to understand what personal data is collected by services they use.
- Helpful for organizations providing clearer and more transparent privacy policies.
- Useful for regulators and auditors reviewing policy clarity and compliance.
- Can be integrated by developers in privacy-first applications.
- Highly relevant in light of GDPR**,** CCPA, and similar data protection regulations.

Future Enhancements

- Multilingual support for simplifying policies written in various languages.
- Integrate OCR to process scanned PDFs.
- Enable downloadable summaries of simplified privacy policies.
- Use larger language models (e.g., GPT variants) for improved accuracy.
- Develop a browser extension to allow real-time simplification of privacy policies on websites.

DATASET DESCRIPTION

This project does not rely on a traditional structured dataset but instead uses real-world privacy policies sourced from public domains. Sample documents include privacy policy templates from small businesses, as well as published policies from major companies such as Google and Apple. The documents are typically in PDF format and range from 2 to 10 pages. Each policy is treated as unstructured text and processed using NLP models for summarization and question answering.

These real-life documents provide authentic legal content that the system simplifies, ensuring practical relevance and robustness in handling diverse language patterns and formats.

CODE

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Privacy Policy Simplifier</title>
    <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
<div class="container mt-4">
    <h2 class="mb-3">🔐 Privacy Policy Simplifier</h2>
    <form method="POST" enctype="multipart/form-data">
        <label class="form-label">Upload PDF File:</label>
        <input type="file" name="pdf" class="form-control mb-3">
        <button type="submit" name="simplify" class="btn btn-primary">Upload and Simplify</button>
    </form>
    <form method="POST" class="mt-4">
        <label>Or Paste Privacy Policy Text:</label>
        <textarea name="raw_text" class="form-control" rows="8">{{ original_text }}</textarea>
        <button type="submit" name="simplify" class="btn btn-success mt-2">Simplify</button>
    </form>
    {% if simplified %}
        <h4 class="mt-4">🧠 Simplified Policy:</h4>
        <textarea class="form-control" rows="6">{{ simplified }}</textarea>
    {% endif %}
```

```
    {% if original_text %}
    <form method="POST" class="mt-4">
        <label>Ask a Question from the Policy:</label>
        <input type="hidden" name="raw_text" value="{{ original_text }}">
        <input type="text" name="question" class="form-control mb-2" placeholder="E.g. Does the app collect email addresses?">
        <button type="submit" class="btn btn-warning">Get Answer</button>
    </form>
    {% endif %}
    {% if answer %}
        <div class="alert alert-info mt-3"><strong>Answer:</strong> {{ answer }}</div>
    {% endif %}
</div>
</body>
</html>
```

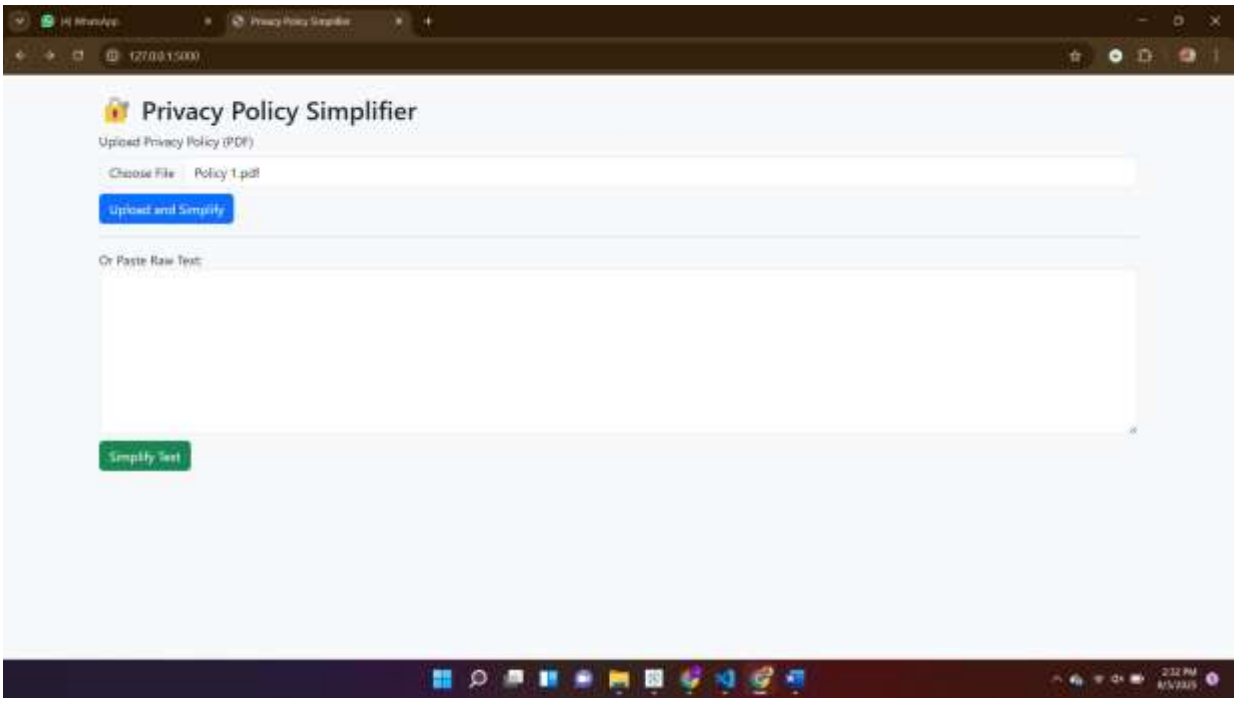
Main.py

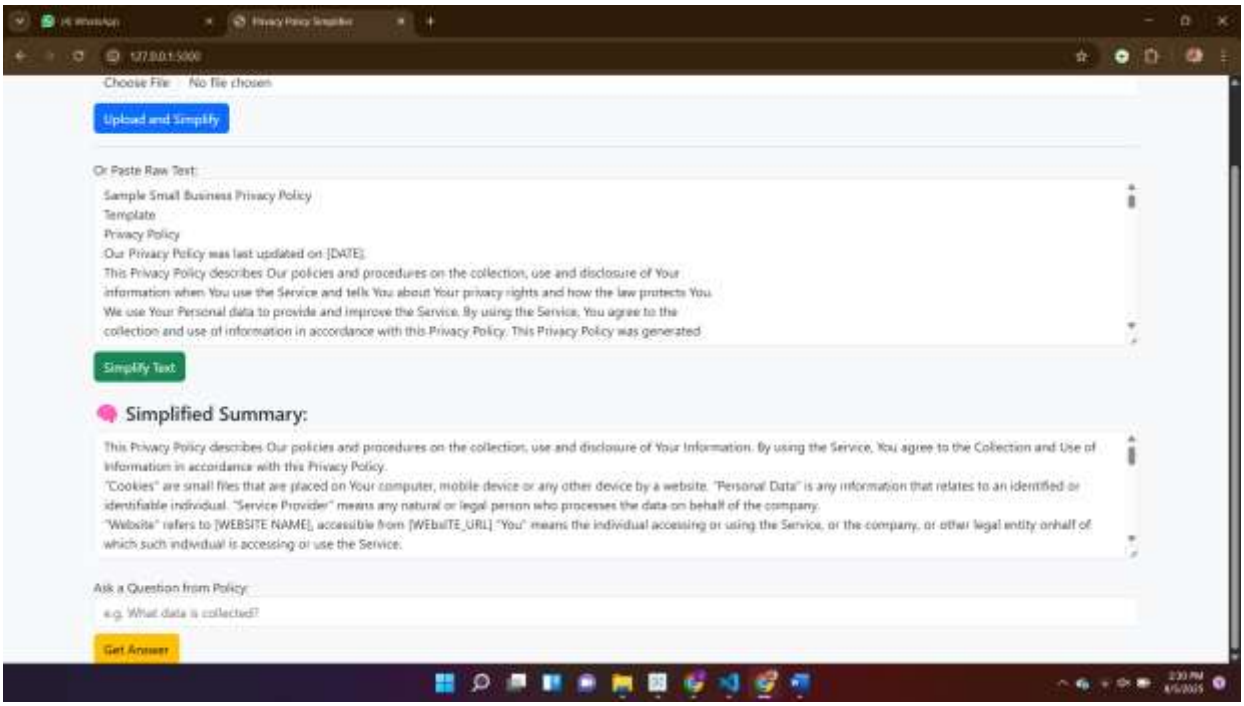
```
import tkinter as tk
from tkinter import filedialog, messagebox, scrolledtext
import fitz  # PyMuPDF
from transformers import pipeline
# Load models
summarizer = pipeline("summarization", model="facebook/bart-large-cnn")
qa_pipeline = pipeline("question-answering", model="distilbert-base-uncased-distilled-squad")
# Extract text from PDF
def extract_text_from_pdf(path):
    text = ""
    with fitz.open(path) as doc:
        for page in doc:
            text += page.get_text()
```

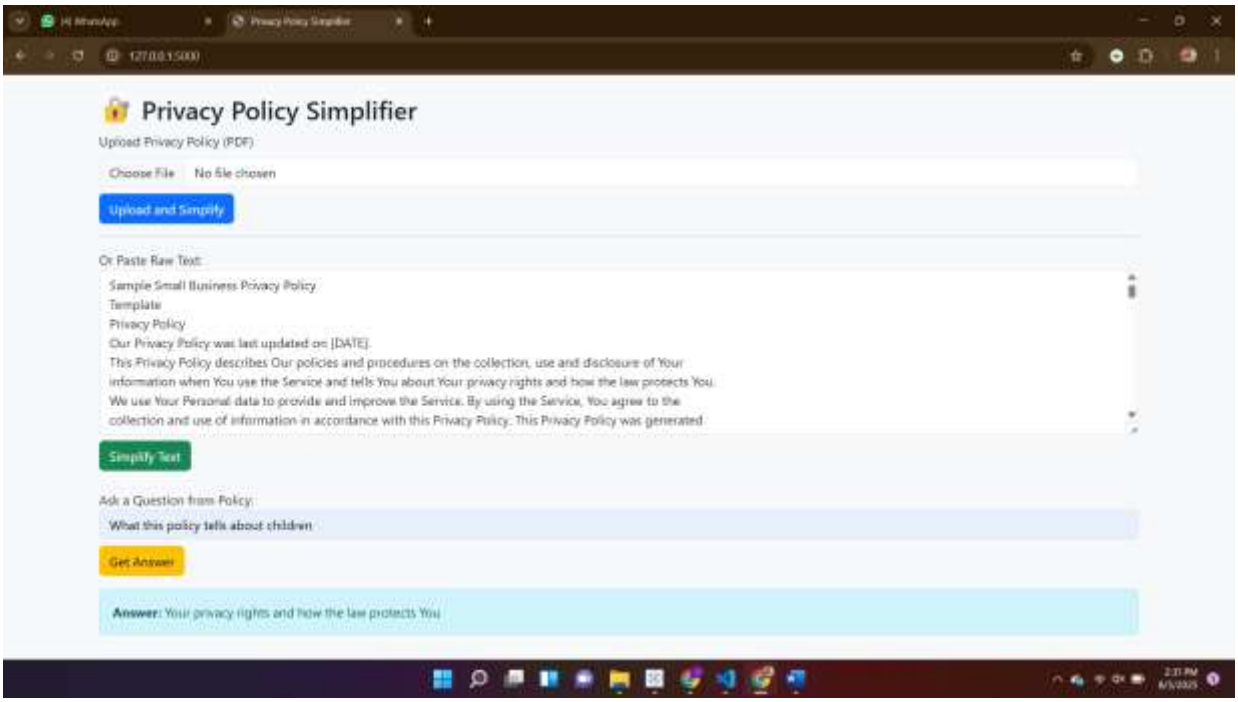
```
    return text
# Simplify text using summarizer
def simplify_text(text):
    chunks = [text[i:i+1024] for i in range(0, len(text), 1024)]
    simplified = []
    for chunk in chunks:
        summary = summarizer(chunk, max_length=130, min_length=30, do_sample=False)[0]['summary_text']
        simplified.append(summary)
    return "\n".join(simplified)
# Answer user questions
def answer_question(question, context):
    result = qa_pipeline({'question': question, 'context': context})
    return result['answer']
# UI Functions
def upload_pdf():
    file_path = filedialog.askopenfilename(filetypes=[("PDF Files", "*.pdf")])
    if file_path:
        text = extract_text_from_pdf(file_path)
        input_text.delete("1.0", tk.END)
        input_text.insert(tk.END, text)
def simplify():
    raw = input_text.get("1.0", tk.END).strip()
    if raw:
        result = simplify_text(raw)
        output_text.delete("1.0", tk.END)
        output_text.insert(tk.END, result)
    else:
        messagebox.showwarning("Warning", "No text to simplify!")
def ask_question():
```

```
    question = question_entry.get()
    context = input_text.get("1.0", tk.END)
    if question.strip() and context.strip():
        answer = answer_question(question, context)
        messagebox.showinfo("Answer", answer)
    else:
        messagebox.showwarning("Warning", "Provide both question and policy text!")
# Create window
root = tk.Tk()
root.title("Privacy Policy Simplifier")
root.geometry("800x700")
# Buttons
tk.Button(root, text="Upload PDF", command=upload_pdf, width=20, bg="lightblue").pack(pady=10)
tk.Label(root, text="OR Paste Your Policy Text Below").pack()
input_text = scrolledtext.ScrolledText(root, wrap=tk.WORD, height=15)
input_text.pack(padx=10, pady=5, fill=tk.BOTH, expand=True)
tk.Button(root, text="Simplify Policy", command=simplify, bg="lightgreen").pack(pady=10)
tk.Label(root, text="Simplified Policy Output:").pack()
output_text = scrolledtext.ScrolledText(root, wrap=tk.WORD, height=10)
output_text.pack(padx=10, pady=5, fill=tk.BOTH, expand=True)
tk.Label(root, text="Ask a Question from the Policy:").pack()
question_entry = tk.Entry(root, width=80)
question_entry.pack(pady=5)
tk.Button(root, text="Get Answer", command=ask_question, bg="orange").pack(pady=10)
root.mainloop()
```

RESULT:


Privacy Policy Simplifier
Upload Privacy Policy (PDF)
Choose File
Policy 1.pdf
Upload and Simplify
Or Paste Raw Text:
Simplify Text


Choose File
No file chosen
Upload and Simplify
Or Paste Raw Text:
Sample Small Business Privacy Policy
Template
Privacy Policy
Our Privacy Policy was last updated on [DATE].
This Privacy Policy describes Our policies and procedures on the collection, use and disclosure of Your information when You use the Service and tells You about Your privacy rights and how the law protects You.
We use Your Personal data to provide and improve the Service. By using the Service, You agree to the collection and use of information in accordance with this Privacy Policy. This Privacy Policy was generated
Simplify Text
Simplified Summary:
This Privacy Policy describes Our policies and procedures on the collection, use and disclosure of Your Information. By using the Service, You agree to the Collection and Use of Information in accordance with this Privacy Policy.
"Cookies" are small files that are placed on Your computer, mobile device or any other device by a website. "Personal Data" is any information that relates to an identified or identifiable individual. "Service Provider" means any natural or legal person who processes the data on behalf of the company.
"Website" refers to [WEBSITE NAME], accessible from [WEbsITE_URL] "You" means the individual accessing or using the Service, or the company, or other legal entity onhalf of which such individual is accessing or use the Service.
Ask a Question from Policy:
e.g. What data is collected?
Get Answer



COMPARATIVE ANALYSIS:

Model Name	Task Type	Strengths	Weaknesses	Accuracy (F1)	Suitability for Project
BART (facebook/bart-large-cnn)	Abstractive Summarization	Human-like summaries, fluent and coherent	Slower, needs GPU for large texts	~44 (CNN/DailyMail)	Best summarizer for legal docs
T5 (t5-small)	Text-to-text, Summarization	Flexible for many NLP tasks, lightweight	Less coherent summaries, fewer details	~36 (CNN/DailyMail)	Moderate performance, fast
PEGASUS	Summarization	Designed specifically for summarization	Very resource-intensive, needs tuning	~43 (XSum benchmark)	High quality, but heavy
DistilBERT	Question Answering	Fast, lightweight, good accuracy	Slightly less accurate than BERT-base	~87 (SQuAD)	Best for real-time QA

BERT (base-uncased)	Question Answering	Very accurate, rich contextual understanding	Slower inference time, higher memory	~78.5 (SQuAD)	Slower, heavier
RoBERTa-base	Question Answering	More accurate than BERT on many tasks	Even larger model size	~79.5 (SQuAD)	High accuracy, less efficient

Why We Use BART for Summarization

- **Handles long and complex legal sentences** better than T5 or GPT-based models.
- **Produces fluent, human-readable summaries** instead of just extracting important lines (as with extractive methods).
- **Fine-tuned on CNN/DailyMail** dataset, which is similar in length and structure to privacy policies.
- Compared to PEGASUS, BART is easier to integrate and slightly lighter.

Why We Use DistilBERT for Question Answering

- **Fast and lightweight**, suitable for real-time web applications without needing GPU support.
- **Retains 95% of BERT's accuracy** while being 40% smaller and 60% faster.
- **Great balance** between performance and computational efficiency for answering user questions on policy content.

Conclusion

The Privacy Policy Simplifier using NLP effectively bridges the gap between complex legal language and user understanding. By leveraging advanced transformer models like BART for summarization and DistilBERT for question answering, the system simplifies lengthy privacy policies and provides clear, accurate responses to user queries. The user-friendly web interface makes it accessible to non-technical users, enhancing digital transparency and informed consent. This project demonstrates the practical application of AI in legal tech and sets a foundation for future tools aimed at improving online privacy awareness and accessibility across diverse user communities.

REFERENCES

1. Vaswani, A., et al. (2017). "Attention is All You Need." *Advances in Neural Information Processing Systems*.
2. Devlin, J., et al. (2019). "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." *arXiv preprint arXiv:1810.04805*.
3. Lewis, M., et al. (2020). "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension." *arXiv preprint arXiv:1910.13461*.
4. Wolf, T., et al. (2020). "Transformers: State-of-the-Art Natural Language Processing." *EMNLP 2020*.
5. Flask Documentation: https://flask.palletsprojects.com/
6. Hugging Face Transformers: https://huggingface.co/transformers/
7. PyMuPDF Documentation: https://pymupdf.readthedocs.io/en/latest/